

PROBLEMES COMPL

$P \equiv$ pot generar un output en temps polinòmic

$NP \equiv$ cal generar un output de forma indeterminada, i la solució es pot comprovar en temps polinòmic.

$coNP \equiv$ Si em dius que no hi ha solució al problema, el puc comprovar ràpidament?



• Un problema decidable $X \in coNP \iff \bar{X} \in NP$

$X \in NP \iff \bar{X} \notin coNP$

Tancament de P , NP i $coNP$

• Unió

a) $A \text{ i } B \text{ a } P \implies A \cup B \in P$

Hi ha una TM M_A que reconeix A en temps polinòmic.

Hi ha una TM M_B que reconeix B en temps polinòmic.

Veiem l'algorisme:

1- String x de mida n

2- Run $M_A(x)$

3- Run $M_B(x)$

4- Si $M_A(x)$ o $M_B(x)$ ho acceptat llavors acceptar, sinó reject

L'algorisme tarda el temps que tarda en comprovar $M_A(x)$ i $M_B(x)$, que és polinòmic

b) $A \text{ i } B \text{ a NP} \Rightarrow A \cup B \in \text{NP}$

Tindrem un verificador V_A i un V_B , també uns testimonis y_A i y_B .
Els verificadors i els testimonis són polinòmics per definició de NP.

Tenim l'entrada x , i es compleix que $(x, y_A) \in V_A$ i $(x, y_B) \in V_B$.

Per comprovar que $A \cup B \in \text{NP}$ crearem testimoni $y = y_A \# y_B$,
i direm que $(x, y) \in V_{A \cup B} \Leftrightarrow (x, y_A) \in V_A \vee (x, y_B) \in V_B$.

- El nou testimoni y és polinòmic perquè s'ha construït a partir de dos testimonis polinòmics i un separador de mida 1.
- El verificador $V_{A \cup B}$ és polinòmic perquè es basa en els verificadors V_A i V_B que són polinòmics.

• Intersecció

a) $A \text{ i } B \text{ a P} \Rightarrow A \cap B \in \text{P}$

Hi ha una TM M_A que reconeix A en temps polinòmic.

Hi ha una TM M_B que reconeix B en temps polinòmic.

Veiem l'algorisme:

1- String x de mida n

2- Run $M_A(x)$

3- Run $M_B(x)$

4- Si $M_A(x)$ i $M_B(x)$ han acceptat llavors acceptem, sinó reject

L'algorisme tarda el temps que tarda en comprovar $M_A(x)$ i $M_B(x)$, que és polinòmic

b) $A \text{ i } B \text{ a P} \Rightarrow A \cap B \in \text{NP}$

Tindrem un verificador V_A i un V_B , també uns testimonis y_A i y_B .

Els verificadors i els testimonis són polinòmics per definició de NP.

Tenim l'entrada x , i es compleix que $(x, y_A) \in V_A$ i $(x, y_B) \in V_B$.

Per comprovar que $A \cap B \in \text{NP}$ crearem testimoni $y = y_A \# y_B$,

i direm que $(x, y) \in V_{A \cap B} \Leftrightarrow (x, y_A) \in V_A \wedge (x, y_B) \in V_B$.

- El nou testimoni y és polinòmic perquè s'ha construït a partir de dos testimonis polinòmics i un separador de mida 1.
- El verificador $V_{A \cap B}$ és polinòmic perquè es basa en els verificadors V_A i V_B que són polinòmics.

Concatenació

$$a) A \text{ i } B \text{ a } P \Rightarrow A \cdot B \in P$$

Signi M_A la TM que decideix A en temps polinòmic, i signi M_B la TM que decideix B en temps polinòmic.

1- Signi x un string de mida n

2- Comparar totes les $n+1$ combinacions per dividir x en la forma $x = yz$

3- Per cada combinació provar que si y és acceptada per M_A i z és acceptada per M_B , llavors $x \in A \cdot B$.

Aquest algorisme té cost polinòmic, perquè crear les $n+1$ combinacions té cost polinòmic, i provar cada combinació també té cost polinòmic, perquè M_A i M_B ho comproven en temps polinòmic.

$$b) A \text{ i } B \text{ a } NP \Rightarrow A \cdot B \in NP$$

Unes tenim un testimoni y_A per A i un y_B per B . També tenim uns verificadors V_A i V_B respectivament. Testimoni i verificador tenen mida/temps polinòmic per definició de NP.

Llavors, existeix una entrada $x = x_1 x_2$ de tal forma que $(x_1, y_A) \in V_A$ i $(x_2, y_B) \in V_B$.

- Testimoni: els testimonis són el parell y_A i y_B que tenen mida polinòmica.
- Verificador: està format per dos verificadors que tenen temps polinòmic.

• Klee

$$a) A \in P \Rightarrow A^* \in P$$

Ho faran amb un algorisme de DP, perquè hi haurà 2 maneres de comprovar en temps exponencial.

$A(i)$ quando si $x_1 \dots x_i$ està en A^* . M_A és una TM que funciona en temps polinomial.

Algorisme:

- Input $x = x_1 \dots x_n$.

- $A = \text{tot false}$

- $A(0) = \text{true}$

- for $i = 1 \dots n$

for $j = 0 \dots i-1$

si $A(j) \wedge M(x_{j+1} - x_i) = 1 \Rightarrow A(i) = \text{true}$

- return $A[n]$

$$b) A \in NP \Rightarrow A^* \in NP$$

Donem el verificador V_A i els testimonis $y_1, \dots, y_k$, i l'entrada $x = x_1 \dots x_k$. Llavors, amb l'indeterminisme a creu una entrada x , i per tot i comprovem que $(x_i, y_i) \in V_A$.

Reduccions en temps polinòmic

a) $E_2 \leq_m^{\uparrow}$ reflexiva? És a dir, $A \leq_m^{\uparrow} A$ per tot llenguatge A ?

Sí, podem oferir la funció identitat: $f(w) = w$, aquesta funció farà que $w \in A \iff f(w) \in A$, i com $f(w) = w$ ja sabem, es redueix a si mateix.

b) $E_2 \leq_m^{\uparrow}$ simètrica? És a dir, si $A \leq_m^{\uparrow} B$, aleshores $B \leq_m^{\uparrow} A$ per a tot llenguatge A i B ?

No, és fals, aquí un contraexemple. Sigui $A = \emptyset$ i $B = \{1\}$.

Per a que $B \leq_m^{\uparrow} A$ caldria que si $w \in B \implies f(w) \in A$, però A no té elements, és impossible que $f(w)$ sigui en A , la reducció falla.

c) $E_2 \leq_m^{\uparrow}$ antisimètrica? És a dir, si $A \leq_m^{\uparrow} B \wedge B \leq_m^{\uparrow} A \implies A = B$

No, és fals, aquí un contraexemple. Sabem que $SAT \leq_m^{\uparrow} CLIQUE$ i

que $CLIQUE \leq_m^{\uparrow} SAT$ perquè tots dos són NP-complets. Però,

$SAT \neq CLIQUE$, SAT està format per fórmules lògiques i $CLIQUE$ per un conjunt de grafes - són diferents.

d) $E_2 \leq_m^{\uparrow}$ transitiva? És a dir, si $A \leq_m^{\uparrow} B \wedge B \leq_m^{\uparrow} C \implies A \leq_m^{\uparrow} C$?

Sí. Veiem la justificació:

1- Com $A \leq_m^{\uparrow} B$ existeix una funció polinòmica $f: w \in A \iff f(w) \in B$

2- Com $B \leq_m^{\uparrow} C$ existeix una funció polinòmica $g: x \in B \iff g(x) \in C$

3- Volem reduir A a C , veiem funció $h: h(w) = g(f(w))$

4- $w \in A \iff f(w) \in B \iff g(f(w)) \in C$, llavors $w \in A \iff h(w) \in C$.

Ceru vs Decid

Si $P=NP$ llavors podem fer preguntes com 'ε-satisfiable(ϕ)',
'clique(G, k)', o 'tiene factor(N, munt)' i les respondria
en temps polinòmic! Podem fer algorismes polinòmics per resoldre
aquests problemes. Tipus 'pregunte a est?'

$\begin{cases} \rightarrow \text{SI}, \dots \\ \rightarrow \text{NO}, \dots \end{cases}$

Teorema de Berman

Un llenguatge Σ^* universali és cada mat que conté ϵ de la forma 1^n , $n \geq 0$.

[Teorema: Si un llenguatge universali és NP-complet $\Rightarrow P=NP$]

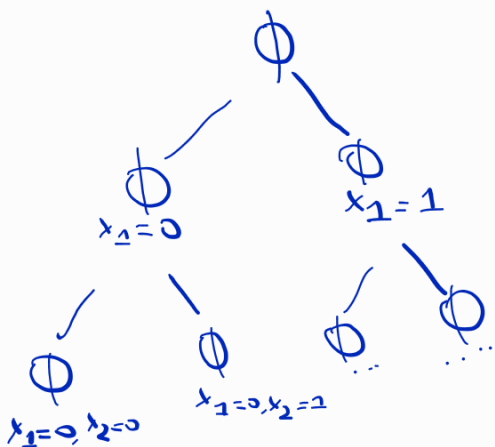
Si L és NP-complet podem reduir $SAT \leq_p L$, això passa si

$$\phi \in SAT \iff f(\phi) \in L$$

$\exists |\phi|=n$ llavors com el llenguatge ϵ universali només podem existir
 $n(n)+1$ instàncies ($1^0 \dots 1^{n(n)}$).

Per demostrar $P=NP$ cal un algorisme polinòmic que resolgui SAT mitjançant f .

Usant l'arbre:



Si tenim dues fórmules proposicionals diferents Φ_A i Φ_B tal que

$f(\Phi_A) = f(\Phi_B)$ llavors $\Phi_A \in SAT \iff f(\Phi_A) \in L \iff f(\Phi_B) \in L \iff \Phi_B \in SAT$

Això vol dir que Φ_A i Φ_B són equivalents, i no n'he solució d'altre temps.

Algoritme

1- Iniciem llista variables booleanes actives $S_0 = \{\Phi\}$

2- Per cada x_i de 1 — m (llista variables)

- Creem S_i buida

- Per cada fórmula Φ en S_{i-1}

- Creem dos fills $\Phi_{false} = \Phi_{x_i=0}$ i $\Phi_{true} = \Phi_{x_i=1}$

- Calculem $v_0 = f(\Phi_{false})$ i $v_1 = f(\Phi_{true})$

- Podem afegir Φ_{false} a $S_i \iff$ no existeix un altre fórmula a S_i que superi el mateix que v_0 . El mateix per v_1

3- Al final obtenim llista de booleanes

4- Si algun llista final acaba en true $\Rightarrow \Phi$ original e sat

Halting problem é NP-Difícil, é NP-Completo?

- NP-Difícil: Se que há é, para demonstrar -lo seria reduzir SAT (NP-Difícil) a HALT, e depois demonstrarem que HALT é NP-Difícil.

$SAT \leq^p HALT$. Depois segue Φ uma fórmula booleana,

Depois $\Phi \in SAT \iff f(\Phi) \in HALT$, e segue $f(\Phi) = \langle M, w \rangle$

↳ Máquina M : (ignora w)

- 1- Buscar todas as possíveis atribuições de verdade para satisfazer Φ
- 2- Comparar qual atribuição satisfaz Φ
- 3- Se satisfaz, a máquina para (HALT)
- 4- Se não, nunca infinita e não para.

Por cada instância positiva de SAT eu tenho uma de HALT, e em caso negativo igual. Então a redução é HALT é NP-difícil.

- NP-Completo: Não há é, sabemos que HALT é indecidível pela teoria, depois HALT se pertence a NP, isto quer dizer que em NP é possível resolver em tempo polinomial se o resposta é sim ou não, e se não, então não se pertence a NP e não é NP-completo.

Unique SAT e DP

- DP (Difference polynomial time): conjunto de linguagens L para as quais existe $L_1 \in NP$ e $L_2 \in coNP$ tal que $L = L_1 \cap L_2$, e com $\overline{L_2} \in NP$ prova-se que L é a diferença de dois conjuntos de NP.
- Unique SAT: determinar se uma ϕ tem 1 e única solução que seja certa.

a) Unique SAT e DP:

Para demonstrar isso vamos encontrar linguagens L_1 e L_2 que formem a definição de Unique SAT, mais a veremos:

$$L_1 = \{ \phi \mid \phi \text{ é satisfatível} \}$$
 equivale a SAT e a ter 1 ou mais soluções.

$$L_2 = \{ \phi \mid \phi \text{ não tem 1 solução que seja certa} \}$$
 mas o outro caso que uma fórmula ϕ não tenha 1 solução

$L_1 \in NP$ por definição, já que equivale a SAT.

Veja L_2 :

$$\text{El negat de } L_2 \text{ é } \overline{L_2} = \{ \phi \mid \phi \text{ tem 2 ou mais soluções que satisfazem } \phi \}$$

Para resolver este problema e achar as soluções x e y , se $x \neq y$, e comprovamos que satisfazem ϕ .

Això també equival a SAT, i com en $L_2 \in NP$ ho veiem
per definició $L_2 \in coNP$.

